

LJ POLYTECHNIC

A MINI PROJECT REPORT

ON

Tic-Tac-Toe

(2024 - 2025)

Java Programming

Submitted by:

Sr. #	Enrollment #	Student Name
1	22012250210410	Ayush Singh
2	22012250210439	Vandan Shah
3	22012250210428	Tithi Brahmshtriya
4	22012250210402	Shubham Thakkar
5	22012250210388	Anvi Shah
6	22012250210378	Sakshi Patel

• Definition

The provided Java code implements a console-based Tic Tac Toe game, allowing for two gameplay modes: Player vs. AI and Player vs. Player. The game features a simple text interface, where players can input their moves, and the game checks for wins or draws after each move.

Structure:

The program is structured around a main class, TicTacToe, which contains the following key components:

Game Board: A 2D array (`char[][] board`) representing the game state, initialized with empty spaces.

Current Player: A variable (`char currentPlayer`) to keep track of which player's turn it is, starting with 'X'.

Game Mode: A boolean (`playerVsAI`) to determine if the game is against an AI or another player.

Game Loop: A loop that continues until the game ends, handling player moves and checking for the game outcome.

Key Functions

Main Method: Sets up the game, asks the user for the game mode, and runs the game loop.

`printBoard()`: Displays the current state of the game board to the console.

`playerMove(Scanner scanner)`: Handles input for player moves, ensuring valid input and updating the board.

`aiMove()`: Generates a random move for the AI player, ensuring it plays in an empty cell.

`switchPlayer()`: Switches the current player after each valid move.

`checkForWin()`: Checks rows, columns, and diagonals for a winning condition.

`checkForDraw()`: Determines if the game has ended in a draw by checking for any empty cells.

Game Modes

Player vs. AI: The AI makes random moves when it's its turn.

Player vs. Player: Both players alternate turns, inputting their moves manually.

Validation

Input validation is implemented in the `playerMove()` method, ensuring moves are made within the bounds of the board and only on empty cells.

The game checks for winning conditions after each move and declares a winner or a draw.

Improvements and Recommendations

AI Enhancement: The AI currently makes random moves. Implementing a more strategic approach (e.g., Minimax algorithm) could improve gameplay.

Input Handling: Adding error handling for non-integer inputs could enhance user experience and prevent crashes.

Game Restart Option: Providing an option to restart the game after it ends would improve usability.

User Interface: Consider implementing a graphical user interface (GUI) for better interaction.

Game Statistics: Track wins, losses, and draws for each player..

• Project Code

```
import java.util.Random;
import java.util.Scanner;

public class TicTacToe {
    private static char[][] board = { { ' ', ' ', ' ' }, { ' ', ' ', ' ' }, { ' ', ' ', ' ' } };
    private static char currentPlayer = 'X';
    private static final Random random = new Random();
    private static boolean playerVsAI;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean gameEnded = false;

        // Ask user to choose the mode (Player vs AI or Player vs Player)
        System.out.println("Choose game mode: ");
        System.out.println("1. Player vs AI ");
        System.out.println("2. Player vs Player ");
        System.out.print("ENTER CHOICE: ");
        int modeChoice = scanner.nextInt(); // Corrected input handling
        playerVsAI = (modeChoice == 1); // If choice is 1, set playerVsAI to true

        while (!gameEnded) {
            printBoard();
            if (currentPlayer == 'X') {
                playerMove(scanner);
            } else {
                if (playerVsAI) {
                    aiMove();
                } else {
                    playerMove(scanner); // In player vs player mode, both take turns
                }
            }
            gameEnded = checkForWin() || checkForDraw();
            if (!gameEnded) {
                switchPlayer();
            }
        }

        printBoard();
        if (checkForWin()) {
            System.out.println("Player " + currentPlayer + " wins!");
        } else {
            System.out.println("The game is a draw!");
        }
    }

    private static void printBoard() {
        System.out.println(" " + board[0][0] + " | " + board[0][1] + " | " + board[0][2]);
        System.out.println("---+---+---");
    }
}
```

```

        System.out.println(" " + board[1][0] + " | " + board[1][1] + " | " + board[1][2]);
        System.out.println("---+---+---");
        System.out.println(" " + board[2][0] + " | " + board[2][1] + " | " + board[2][2]);
    }

    private static void playerMove(Scanner scanner) {
        int row, col;
        while (true) {
            System.out.println("Player " + currentPlayer + ", enter your move:");

            // Ask for row
            System.out.print("Enter row (1, 2, or 3): ");
            row = scanner.nextInt() - 1; // Converting to zero-based index

            // Ask for column
            System.out.print("Enter column (1, 2, or 3): ");
            col = scanner.nextInt() - 1; // Converting to zero-based index

            // Validating move
            if (row >= 0 && row < 3 && col >= 0 && col < 3 && board[row][col] == ' ') {
                board[row][col] = currentPlayer;
                break; // Exit the loop if the move is valid
            } else {
                System.out.println("This move is not valid. Try again.");
            }
        }
    }

    private static void aiMove() {
        int row, col;
        while (true) {
            row = random.nextInt(3); // Random row
            col = random.nextInt(3); // Random column

            // Check if the chosen cell is empty
            if (board[row][col] == ' ') {
                board[row][col] = currentPlayer;
                break; // Exit the loop if the move is valid
            }
        }
        System.out.println("AI (Player " + currentPlayer + ") makes a move.");
    }

    private static void switchPlayer() {
        currentPlayer = (currentPlayer == 'X') ? 'O' : 'X';
    }

    private static boolean checkForWin() {
        // Check rows
        for (int i = 0; i < 3; i++) {
            if (board[i][0] == currentPlayer && board[i][1] == currentPlayer && board[i][2] ==

```

```

currentPlayer) {
    return true;
}
}
// Check columns
for (int i = 0; i < 3; i++) {
    if (board[0][i] == currentPlayer && board[1][i] == currentPlayer && board[2][i] ==
currentPlayer) {
        return true;
    }
}
// Check diagonals
if (board[0][0] == currentPlayer && board[1][1] == currentPlayer && board[2][2] ==
currentPlayer) {
    return true;
}
if (board[0][2] == currentPlayer && board[1][1] == currentPlayer && board[2][0] ==
currentPlayer) {
    return true;
}
return false;
}

private static boolean checkForDraw() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == ' ') {
                return false;
            }
        }
    }
    return true;
}
}

```

• Output

PS C:\Users\test\Desktop\JAVA @ MINI-PROJECT> javac TicTacToe.java

• PS C:\Users\test\Desktop\JAVA @ MINI-PROJECT> java TicTacToe

Choose game mode:

1. Player vs AI

2. Player vs Player

ENTER CHOICE: 1

```

  |   |
  +---+
  |   |
  +---+
  |   |

```

Player X, enter your move:

Enter row (1, 2, or 3): 1

Enter column (1, 2, or 3): 3

```

  |   | X
  +---+
  |   |
  +---+
  |   |

```

AI (Player 0) makes a move.

```

  |   | X
  +---+
  |   |
  +---+
  |   | O

```

Player X, enter your move:

Enter row (1, 2, or 3): 1

Enter column (1, 2, or 3): 1

```

X |   | X
  +---+
  |   |
  +---+
  |   | O

```

AI (Player 0) makes a move.

```

X |   | X
  +---+
  | O |
  +---+

```

		O
--	--	---

Player X, enter your move:
Enter row (1, 2, or 3): 3
Enter column (1, 2, or 3): 1

X		X
	O	
X		O

AI (Player 0) makes a move.

X		X
	O	O
X		O

Player X, enter your move:
Enter row (1, 2, or 3): 1
Enter column (1, 2, or 3): 2

X	X	X
	O	O
X		O

- **Conclusion**

Tic-Tac-Toe serves as a timeless game that embodies strategic thinking and quick decision-making. Its simplicity allows players of all ages to engage easily, yet it also offers depth in strategy, particularly as players develop advanced techniques to outmaneuver their opponents. Each game, whether resulting in a win, loss, or draw, reinforces critical skills such as planning, foresight, and adaptability. Moreover, Tic-Tac-Toe often serves as an introductory platform for concepts in game theory and artificial intelligence, making it a valuable tool in both educational and recreational contexts. Ultimately, the game's enduring popularity lies in its balance of accessibility and challenge, ensuring it remains a staple in casual play and competitive settings alike.